

Module 6 – Mer-stack – JavaScript Essential and Advanced

Theory Assignment

Question 1: What is JavaScript? Explain the role of JavaScript in web development.

Ans:- JavaScript is a programming language used to make web pages interactive and dynamic.

It helps add functionality like button clicks, form validation, animations, sliders, popups, and API data fetching.

Role in Web Development

- **HTML → structure**
- **CSS → styling**
- **JavaScript → functionality and behavior**

Example:

When you click a button and text changes on the page, JavaScript performs that action.

Question 2: How is JavaScript different from other programming languages like Python or Java?

Ans:-

JavaScript	Python	Java
Mainly used for web development	Used for AI, backend, automation	Used for software & Android apps
Runs in browser	Runs on server/system	Compiled language
Dynamic typing	Dynamic typing	Static typing
Used with HTML & CSS	General-purpose	Object-oriented

Question 3: Discuss the use of <script> tag in html. How can you link an external js file to an html document ?

Ans:- The <script> tag is used to include JavaScript in HTML, and external JavaScript files are linked using the src attribute.

=====

Variables and Data Types

Question 1: What are variables in JavaScript? How do you declare a variable using var, let, and const?

Ans:- Variables are containers used to store data values.

1) var

Old way to declare variables.

→ var age = 22;

2) let

Modern way, value can be changed.

→ let city = "Surat";
city = "Ahmedabad";

3) const

Used for **fixed values, cannot be changed.**

→ const country = "India";

Question 2: Explain the different data types in JavaScript. Provide examples for each.

Ans:-

1) Number:-

→ A number data type is used to **store numeric values like integers and decimals.**

Example:-

let age = 22;

2) String:-

→ A string is used to **store text or characters.**

Example:-

let name = "Mohit";

3) Boolean:-

→ A boolean data type stores **only two values: true or false.**

Example:-

```
let is Passed = true;
```

4) Undefined:-

→ When variable is declared but no value is assigned.

Example:-

```
let x;
```

5) Null

→ When variable is intentionally set as empty.

Example:-

```
let data = null;
```

6) Object:-

→ An object is a collection of related data stored as key–value pairs.

Example:-

```
let person = {  
  name: "Mohit",  
  age: 22  
};
```

7) Array:-

→ An array is used to store multiple values in a single variable.

Example:

```
let colors = ["red", "blue", "green"];
```

Question 3: What is the difference between undefined and null in JavaScript?

Ans:- . Undefined

When variable is declared but no value is assigned.

```
→let x;  
console.log(x); // undefined
```

2. Null

When variable is intentionally set as empty.

→let x = null;

Difference

- undefined → no value assigned
 - null → intentionally empty value
-

JavaScript Operators

Question 1: What are the different types of operators in JavaScript? Explain with examples.

Ans:-

• Arithmetic operators:-

- These are used for mathematical calculations.
- Operators: + , - , * , / ,

• Assignment operators:-

- These are used to assign values to variables.
- Operators: = , += , -= , *= , /=

• Comparison operators:-

- These are used to compare two values and return true or false.
- Operators: == , === , != , > , < , >= , <=

• Logical operators:-

- These are used to combine conditions.
- Operators: && , || , !

Question 2: What is the difference between ==and ===in JavaScript?

Ans:-

1) == (Loose Equality)

It compares only the value, not the data type.

```
console.log(10 == "10"); // true
```

2) === (Strict Equality)

It compares both value and data type.

```
console.log(10 === "10"); // false
```

Because one is a **number** and the other is a **string**.

=====

Control Flow (If-Else, Switch)

Theory Assignment

Question 1: What is control flow in JavaScript? Explain how if-else statements work within example.

Ans:- Control flow means the order in which JavaScript code runs based on conditions, loops, and function calls.

It helps the program decide which block of code should execute.

if-else Statement

The if-else statement is used to **check a condition**.

- If the condition is **true**, the if block runs.
- If the condition is **false**, the else block runs.

Example:-

```
let age = 18;
if (age >= 18) {
  console.log("Eligible for voting");
} else {
  console.log("Not eligible for voting");
}
```

Question 2: Describe how switch statements work in JavaScript. When should you use a switch statement instead of if-else?

Ans:-

❖ Use switch when:

- We are checking **one variable** against many fixed values.
- Conditions are **simple equality checks**.
- You want **clean and readable code**.

❖ **Use if-else when:**

- You need **range or complex conditions**
- Multiple variables are involved
- Logical operators (&&, ||, <, >) are used
- • switch → best for **fixed values** (menu, options, roles, days)
- • if-else → best for **complex logic and conditions**

Loops (For, While, Do-While)

Theory Assignment

Question 1: Explain the different types of loops in JavaScript (for, while, do-while). Provide a basic example of each.

Ans:- .

1. For loop:-

➔ Used when we know **how many times we want to run the loop.**

2. While loop:-

➔ Used when we **don't know exact number of iterations**, but depend on a condition.

3. Do-While loop:-

➔ This loop runs **at least one time**, even if condition is false.

• **Question 2: What is the difference between a while loop and a do-while loop?**

Ans:-

Feature	While loop	Do-while-loop
Condition check	Before loop runs	After loop runs
Execution	May run 0 times	Runs at least 1 time
Syntax ending	No semicolon at end	Ends with ;

Functions

Theory Assignment

- **Question 1: What are functions in JavaScript? Explain the syntax for declaring and calling a function.**

Ans:- A function in JavaScript is a **block of reusable code designed** to perform a specific task. Functions **help make code organized, reusable, and easy to maintain.**

- **Question 2: What is the difference between a function declaration and a function expression?**

Ans:- Function declarations are **hoisted**, while function expressions are not **hoisted in the same way.**

Function Declaration	Function Expression
Declared using function keyword with a name	Function is stored in a variable
Can be called before declaration (hoisting)	Cannot be called before definition
Has a fixed function name	Can be anonymous (without name)

- **Question 3: Discuss the concept of parameters and return values in functions.**

Ans:-

Parameters:-

Parameters are **values passed into a function when it is defined.** They act like **variables inside the function.**

Return Value:-

A **return value** is the value a function sends back using the return keyword.

Arrays Theory

Assignment

- **Question 1: What is an array in JavaScript? How do you declare and initialize an array?**

Ans:- An array in JavaScript is a special variable used to **store multiple values in a single variable**. Arrays can hold numbers, strings, objects, and other data types.

- **Question 2: Explain the methods push(), pop(), shift(), and unshift() used in arrays.**

Ans:-

Method	Description
push()	Adds element at the end of array
pop()	Removes element from the end
shift()	Removes element from the beginning
unshift()	Adds element at the beginning

Objects Theory

Assignment

• **Question 1: What is an object in JavaScript? How are objects different from arrays?**

Ans:- An **object** in JavaScript is a collection of **key-value pairs** used to store related data in one variable.

Each value is stored with a **property name (key)** and its value.

Object	Array
Stores data in key-value pairs	Stores data as ordered list
Accessed using property name (key)	Accessed using index number
Uses { } curly braces	Uses [] square brackets
Can store properties with names	Stores values by position

• **Question 2: Explain how to access and update object properties using dot notation and bracket notation**

Ans:- 1) Dot Notation (.)

Dot notation is the **simplest way to access object properties**.

Key Points:

- ✓ Easy to write
- ✓ Easy to read
- ✓ Works with fixed property names

Update Property:

```
let student = {  
  name: "Mohit",  
  age: 22,  
  city: "Surat"  
};  
  
student.age = 23;  
console.log(student.age); // 23
```

2. Bracket Notation ([])

Bracket notation uses **square brackets** and **property name** as a string.

Key Points:

- ✓ Useful when property name is dynamic
- ✓ Can access properties with spaces/special characters

JavaScript Events

Theory Assignment

- **Question 1: What are JavaScript events? Explain the role of event listeners.**

Ans:- JavaScript events are actions or occurrences that happen in the browser, which JavaScript can detect and respond to.

Examples of events:

- **click** → when user clicks a button
- **mouseover** → when mouse moves over an element
- **key down** → when keyboard key is pressed
- **submit** → when form is submitted
- **load** → when page finishes loading

❖ **Role of Event Listeners:**

- ➔ An **event listener** waits for a specific event to happen and then runs a function.
- ➔ It helps make web pages **interactive**.

E.X

```
button.addEventListener("click", function() {  
    console.log("Button clicked!");  
});
```

- **Question 2: How does the add Event Listener() method work in JavaScript? Provide an example**

Ans:- The addEventListener() method attaches an event handler to an HTML element without overwriting existing events.

E.X element.addEventListener("event", function);

Parameters:

1. **event** → event name (click, key down, mouseover, etc.)
2. **function** → code to execute when event happens

DOM Manipulation

Theory Assignment

• **Question 1: What is the DOM (Document Object Model) in JavaScript? How does JavaScript interact with the DOM?**

Ans:- The DOM (Document Object Model) is a programming **interface** for HTML and XML documents. **It represents the web page as a tree structure** of objects, where each HTML element becomes an object.

JavaScript uses the DOM to:

- Access HTML elements
- Change content of elements
- Change styles (CSS)
- Add or remove elements
- Handle user events (click, submit, etc.)

• **Question 2: Explain the methods get Element By Id(), get Elements By Class Name(), and query Selector() used to select elements from the DOM.**

Ans:-

1) getElementById()

Selects one element by its id.

Example:

```
let element = document.getElementById("title");  
console.log(element);
```

✓ **Returns single element**

✓ **ID must be unique**

2) `getElementsByClassName()`

Selects all elements with same class name.

Example:

```
let items = document.getElementsByClassName("box");  
console.log(items);
```

✓ Returns HTML Collection

✓ Can select multiple elements

3) `querySelector()`

Selects the first matching element using CSS selector.

Example:

```
let element = document.querySelector(".box");  
console.log(element);
```

✓ Uses CSS selectors (#id, .class, tag)

✓ Returns first matching element

Example:

```
document.querySelector("#title");  
document.querySelector(".box");  
document.querySelector("p");
```

JavaScript Timing Events (setTimeout, setInterval)

Theory Assignment

• **Question 1:** Explain the `setTimeout()` and `setInterval()` functions in JavaScript. How are they used for timing events?

Ans:-

1. `setTimeout()`

Executes code once after a specified delay.

E.X `setTimeout(function() {
 console.log("Hello after 2 seconds");
}, 2000);`

2. setInterval()

Executes code **repeatedly** after fixed interval.

E.X setInterval(function() {
 console.log("Runs every 1 second");
}, 1000);

• **Question 2: Provide an example of how to use setTimeout() to delay an action by 2 seconds.**

Ans:- console.log("Start");
setTimeout(function() {
 console.log("This message appears after 2 seconds");
}, 2000);
console.log("End");

JavaScript Error Handling

Theory Assignment

• **Question 1: What is error handling in JavaScript? Explain the try, catch, and finally blocks with an example.**

Ans:- Error handling is used to manage runtime errors so the program does not stop unexpectedly.

1. try

Contains code that may cause error.

2. catch

Handles error if it occurs.

3. finally

Runs always, whether error occurs or not.

• **Question 2: Why is error handling important in JavaScript applications?**

Ans:-

- Prevents **program crash**
- Handles unexpected **errors safely**
- Improves **user experience**
- Makes **debugging easier**
- Keeps **application running smoothly**